AWS KMS

SECRETS MANAGER

SOLUTIONS ARCHITECT

AWS KMS & Secrets Manager

A comprehensive study guide covering key management architecture, encryption patterns, secrets rotation, compliance auditing, best practices, pitfalls, and cost optimization.

April 3, 2026 · 17 sections · Solutions Architect Reference

CONTEXT

Data breaches expose billions of records annually, with stolen credentials and unencrypted data causing 80% of security incidents. Traditional approaches — manual key management, secrets in config files, hardcoded credentials — create vulnerabilities that attackers exploit. AWS KMS and Secrets Manager address these through centralized key management, automatic encryption, credential rotation, and fine-grained access control.

TABLE OF CONTENTS

01 AWS Key Management Service (KMS)

02 KMS Architecture & Key Hierarchy

03 KMS API Operations

04 Key Policies

05 AWS Secrets Manager

06 Compliance & Audit

07 Key Management Best Practices

08 Cost Optimization

09 Tips & Best Practices

10 Pitfalls & Remedies

11 Summary & Key Takeaways

01 AWS Key Management Service (KMS)

1. Customer Master Keys (CMKs)

- Logical representation of an encryption key
- Never leaves KMS unencrypted

- Stored in FIPS 140-2 Level 2 HSMs
- Cannot be exported

2. CMK Types

Type	Created by	Cost	Rotation	Control	Notes
AWS Managed CMK	AWS services automatically	**Free**	Automatic — 3 years	None (cannot change key policy)	Named `aws/service-name`, e.g. `aws/s3`
Customer Managed CMK	Customer explicitly	**$1/month per key**	Optional — annual	Full control over policies	Recommended for production; can be scheduled for deletion
AWS Owned CMK	AWS (shared across accounts)	**Free**	AWS-managed	No visibility or control	Example: DynamoDB default encryption

3. Key Material Origin

Origin	Description	Use Case
KMS (Default)	Key material generated in KMS HSMs	Highest security — recommended
External	Import your own key material	Compliance requirements; you are responsible for securing the original; can set expiration date
Custom Key Store (CloudHSM)	Keys stored in dedicated CloudHSM cluster	Single-tenant HSMs; full control; higher cost ($1,000+/month)

4. Key States

Enabled
Available for encryption/decryption

Disabled
Temporarily unavailable; can re-enable

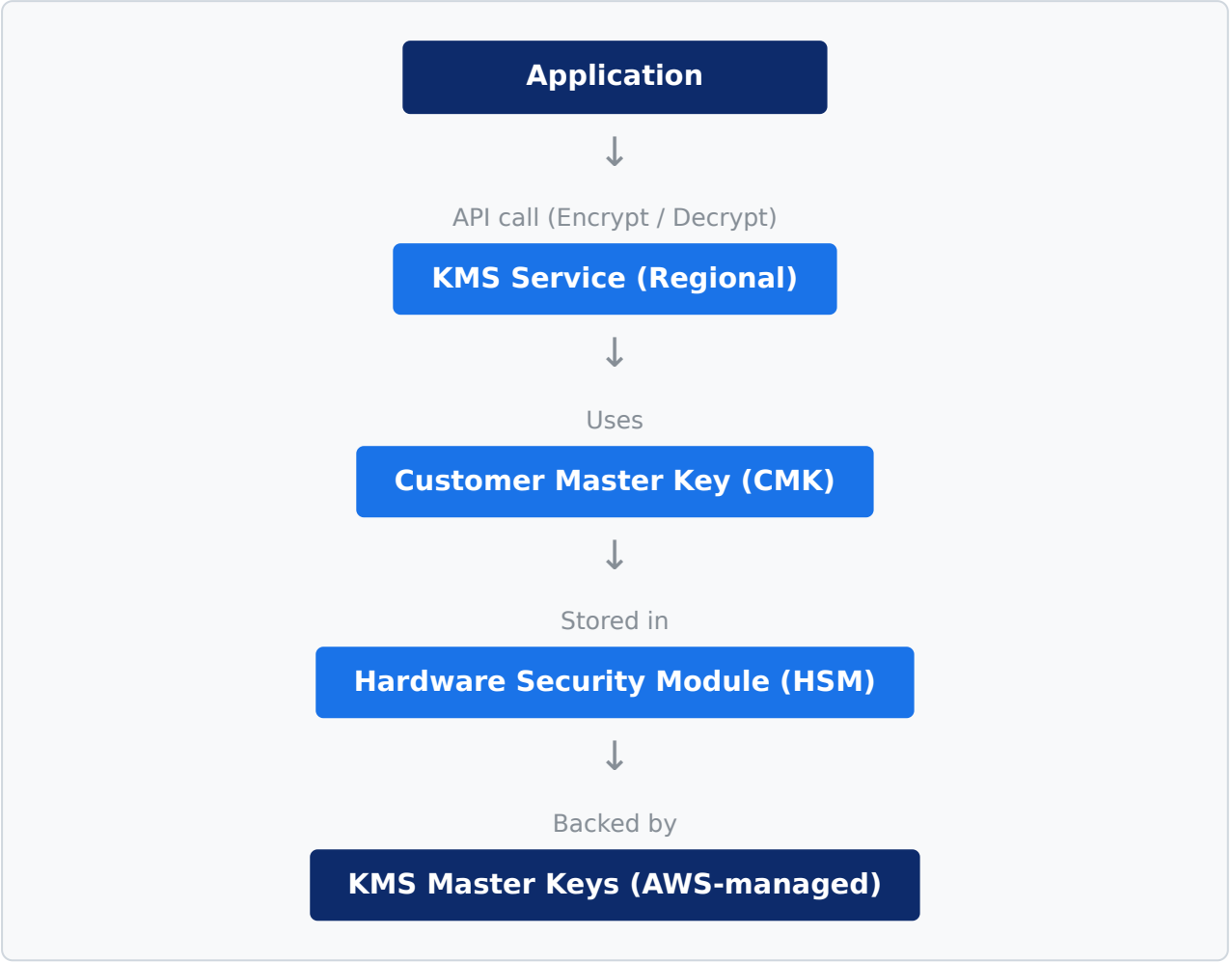
Pending Deletion
7–30 day waiting period; cancellable

Pending Import
Waiting for external key material

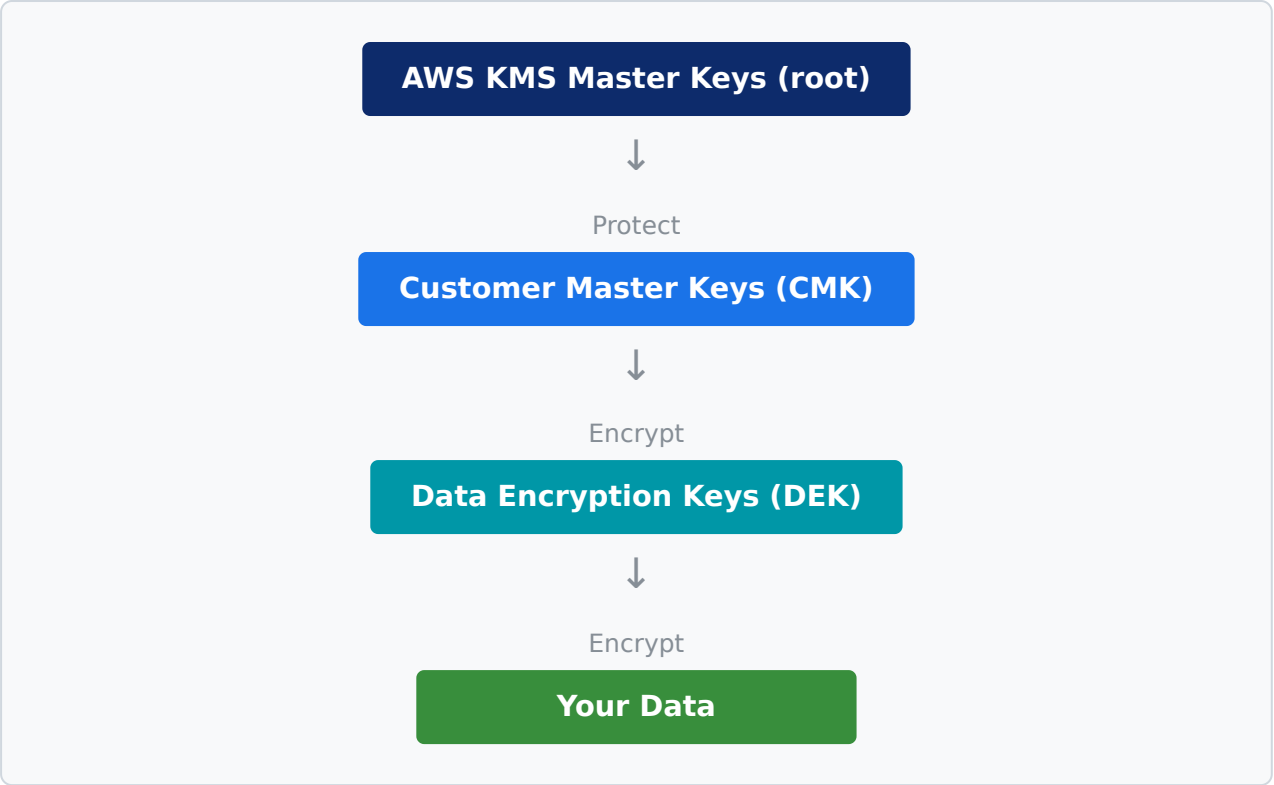
Unavailable
Key material expired or deleted

02 KMS Architecture & Key Hierarchy

Architecture Overview



Key Hierarchy (Envelope Encryption)



WHY ENVELOPE ENCRYPTION?

Encrypting large data directly with a CMK is limited to 4 KB. Envelope encryption generates a local DEK for each piece of data, encrypts the DEK with the CMK, and stores the encrypted DEK alongside the ciphertext — eliminating the size limit and dramatically reducing KMS API calls.

03 KMS API Operations

#	Operation	Input / Output	Primary Use Case
1	**Encrypt**	Plaintext (max 4 KB) → Ciphertext	Small data: passwords, tokens
2	**Decrypt**	Ciphertext → Plaintext	Decryption with automatic CMK detection
3	**GenerateDataKey**	Returns plaintext DEK + encrypted DEK	Envelope encryption; most common for large data
4	**GenerateDataKeyWithoutPlaintext**	Returns only encrypted DEK	Pre-generate keys for future use
5	**ReEncrypt**	Decrypt with one CMK, re-encrypt with another — entirely within KMS	Key migration, key rotation

04 Key Policies

KMS Key Policies are **resource-based policies** attached directly to each CMK. They are the primary access control mechanism for KMS keys.

Key Policy Best Practices

- Separate administrative and usage permissions
- Apply least-privilege principle
- Include condition keys for additional security
- Document each statement using `Sid`
- Test policies in a staging environment before production
- Do not grant blanket `kms:*` to applications
- Do not delete the default root-account statement (locks out IAM entirely)

05 AWS Secrets Manager

Purpose & Architecture

- Store secrets securely (encrypted at rest with KMS)
- Automatic rotation — database passwords, API keys
- Audit access via CloudTrail logging
- Native integration with AWS services
- Full versioning with rollback capability

Secret Types

Type	Examples	Rotation Support

Database Credentials	RDS MySQL/PostgreSQL/Aurora, Redshift, DocumentDB	Built-in automatic rotation via Lambda function
API Keys	Third-party API credentials, OAuth tokens, service account passwords	Custom rotation logic
SSH Keys	Private keys, certificate authorities, PEM files	Custom rotation logic
Generic Secret	Key-value pairs, JSON documents, binary data	Custom rotation logic

Secret Structure (Example)

```
{
    "SecretName": "prod/myapp/database",
    "SecretString": {
        "username": "admin",
        "password": "randomly-generated-password",
        "engine": "mysql",
        "host": "mydb.cluster-abc123.us-east-1.rds.amazonaws.com",
        "port": 3306,
        "dbname": "myapp"
    },
    "VersionStages": ["AWSCURRENT"],
    "VersionId": "uuid-1234-5678",
    "CreatedDate": "2025-01-15T10:30:00Z"
}
```

Version Stages

Stage	Description
`AWSCURRENT`	Current active version used by applications
`AWSPENDING`	New version being rotated in
`AWSPREVIOUS`	Previous version; available for rollback

Secret Rotation

- Automatic or manual scheduling (configurable interval in days)
- Zero-downtime rotation — applications read `AWSCURRENT` at all times
- Automatic retry on failure
- Integrated CloudWatch monitoring for rotation events

06 Compliance & Audit

CloudTrail — KMS Logging

Every KMS API call is logged to CloudTrail with the following attributes:

Attribute	Detail
Who	IAM user / role ARN
When	Timestamp (UTC)
What	API operation (Encrypt, Decrypt, GenerateDataKey, etc.)
Which Key	CMK ARN
Source IP	Caller's IP address
Result	Success or failure

Audit Use Cases

- Who accessed which secrets, and when?
- Detect unauthorized decryption attempts
- Compliance reporting (HIPAA, PCI DSS)
- Incident investigation
- Key usage analytics

CloudWatch Metrics

- `NumberOfNotEncryptedObjects` (S3)
- KMS API call rates
- Failed operations count
- Key state change events

Secrets Manager — CloudTrail Events

Event	Meaning
`GetSecretValue`	Who retrieved the secret
`RotateSecret`	Rotation lifecycle events
`PutSecretValue`	Secret value updates
`DeleteSecret`	Deletion attempts

07 Key Management Best Practices

Enterprise Key Hierarchy

Level	Keys	Managed By	Standard
Organizational	Root Keys (AWS KMS Master Keys)	AWS	FIPS 140-2 Level 3; protect all customer keys
Application	Per-application CMKs: `app-prod-encryption-key` `app-dev-encryption-key` `analytics-encryption-key`	Customer	One key per application/workload
Environment	Production CMK, Staging CMK, Development CMK	Customer	Strict access in production; broader in dev
Purpose-Based	DB encryption key, S3 key, Secret key, Backup key	Customer	Granular separation per use case

Benefits of Key Separation

- Blast radius limitation — a compromised key only exposes one workload
- Clear ownership and accountability
- Easier compliance auditing per application
- Granular access control per team/service
- Cost tracking per application via tagging

08 Cost Optimization

KMS Cost Structure

Item	Cost
Customer-managed CMK storage	**$1.00 / month per key**
First 20,000 API requests / month	**Free**
API requests beyond 20,000	$0.03 per 10,000 requests

Cost Reduction Strategies

#	Strategy	Description	Potential Saving
1	**Use AWS-Managed Keys When Possible**	Free; automatic rotation; suitable for non-compliance workloads; trade-off: less control	**$1/month per key avoided**
2	**Consolidate Keys**	One key per application, not per resource. 100 S3 buckets = 1 key (not 100)	**~$99/month on 100-bucket scenario**
3	**Cache Data Keys**	Generate DEK once; reuse for multiple objects; reduces `GenerateDataKey` calls	**Up to 90% of API costs**
4	**Client-Side Caching (Encryption SDK)**	AWS Encryption SDK caches plaintext data keys automatically with configurable TTL	**Significant reduction**

5	**Batch Operations**	Encrypt multiple items with the same data key; single `GenerateDataKey` call	**10,000 records: $0.30 → $0.000003**

09 Tips & Best Practices

Key Management

TIP 1 — KEY ROTATION

Always enable automatic annual rotation for customer-managed keys. Transparent to applications; maintains compliance.

TIP 2 — KEY ALIASES

Reference keys by alias (`alias/my-app-key`) instead of key ID — allows key replacement without code changes.

TIP 3 — ENCRYPTION CONTEXT

Add encryption context for additional security and audit clarity. Prevents misuse of encrypted data without additional cost.

TIP 4 — ENVIRONMENT SEPARATION

Use different CMKs for production, staging, and development — limits blast radius and simplifies auditing.

TIP 5 — KEY DESCRIPTIONS

Add clear descriptions to CMKs so teams understand key usage and ownership at a glance.

Secrets Management

TIP 6 — CACHE SECRETS

Cache secrets with a 1-hour TTL — reduces API calls by ~99%, lowering costs and latency.

TIP 7 — AUTOMATIC ROTATION

Enable automatic rotation for RDS/Redshift credentials — zero-downtime password changes on schedule.

TIP 8 — TAG SECRETS

Tag secrets with *Environment*, *Application*, *Owner* — enables cost tracking and policy-based access control.

TIP 9 — CROSS-ACCOUNT ACCESS

Use resource policies for cross-account access — simpler than IAM role assumption for most scenarios.

TIP 10 — MONITOR ACCESS

Enable CloudTrail logging; alert on unusual access patterns to detect potential breaches early.

Security

TIP 11 — NO HARDCODED CREDENTIALS

Always use Secrets Manager or Parameter Store. Hardcoded credentials in code or configs are the primary breach vector.

TIP 12 — LEAST PRIVILEGE

Grant minimum KMS permissions: `kms:Decrypt` only for applications — never `kms:*`.

TIP 13 — CLOUDTRAIL COVERAGE

Ensure CloudTrail is enabled in all regions for a complete audit trail of key usage.

TIP 14 — MANUAL ROTATION REMINDERS

If not using automatic rotation, set quarterly calendar reminders to prevent credential staleness.

TIP 15 — DISASTER RECOVERY TESTING

Regularly test restoring encrypted data with backup CMKs to verify recovery procedures work as expected.

10 Pitfalls & Remedies

Pitfall 1 — Accidental Key Deletion

Problem

A CMK is scheduled for deletion. Encrypted data becomes permanently inaccessible after the 7–30 day waiting period expires.

Why It Happens

- Accidental deletion via console or API
- Automation / cleanup scripts without proper checks
- Misunderstanding of the deletion waiting period

Impact

- All data encrypted with the key is permanently inaccessible
- S3 objects unreadable; EBS volumes unmountable; RDS databases inaccessible
- Complete data loss if no backups exist with a different key
- Business disruption, compliance violations, regulatory exposure

Real Example Timeline

- **Day 1:** Administrator accidentally schedules key for deletion
- **Day 7–30:** Waiting period — can still cancel deletion
- **Day 31:** Key permanently deleted
- **Result:** 10 TB of encrypted S3 data permanently inaccessible
- **Recovery:** Impossible — data lost forever
- **Cost:** Millions in data loss, recovery efforts, and reputational damage

Prevention

- Implement an approval workflow for key deletion operations
- Tag critical keys with `Environment=Production`
- Enable CloudWatch alarms for `ScheduleKeyDeletion` API events
- Automated cancellation rule for production keys
- Regular audits of key status across all regions
- Security training on deletion consequences

- Require MFA for key deletion operations

Pitfall 2 — Secrets Manager Permission Issues

Problem

Applications are unable to retrieve secrets due to IAM permission misconfigurations.

Why It Happens

- Missing `secretsmanager:GetSecretValue` permission
- Missing `kms:Decrypt` permission for the CMK used to encrypt the secret
- Resource ARN mismatch in the IAM policy
- VPC endpoint access not configured
- Condition keys inadvertently blocking access

Impact

- Application startup failures
- Database connection errors and service outages
- Difficult to debug — permission errors can be opaque
- Development delays during deployment

Prevention

- Use the IAM Policy Simulator before deployment
- Test with actual role credentials, not admin credentials
- Document required permissions per service in a runbook
- Create reusable IAM policy templates for common access patterns
- Automate permission validation in CI/CD pipelines
- Monitor CloudTrail for `AccessDenied` errors and alert immediately

Pitfall 3 — Rotation Failures

Problem

Automatic secret rotation fails, leaving old passwords active or breaking application database access.

Why It Happens

- Lambda rotation function errors (bugs, unhandled exceptions)
- Database connection issues during the rotation window
- Insufficient Lambda execution permissions
- Network connectivity problems — Lambda in VPC without NAT Gateway
- Database max-connections limit reached
- Lambda timeout (default 3s too short for slow DBs)

Impact

- Old passwords not rotated — ongoing security risk
- Applications lose database access if `AWSPENDING` version is incomplete
- Service outages; compliance violations
- Manual intervention required

Example Failure Sequence

1. Lambda function is invoked by Secrets Manager
2. Lambda attempts to connect to database to create new password
3. Database connection times out (Lambda in VPC without NAT)
4. Rotation fails after 15-minute Lambda timeout
5. Secret stuck in `AWSPENDING` state
6. Applications continue using `AWSCURRENT` (old password)
7. 30-day rotation compliance requirement is violated

Prevention

- Test rotation manually (`rotate-secret --rotate-immediately`) before enabling automatic mode
- Ensure Lambda has proper VPC configuration with internet access (NAT Gateway or VPC endpoints)
- Monitor CloudWatch Logs for rotation Lambda errors

- Set up CloudWatch alarms specifically for rotation failure events
- Increase Lambda timeout to at least 5 minutes for database rotation functions
- Document rotation troubleshooting runbook and train on-call engineers
- Maintain backup admin credentials in a separate, manually rotated secret

11 Summary & Key Takeaways

AWS KMS and Secrets Manager provide enterprise-grade encryption and secrets management, eliminating the complexity and risk of manual key management. KMS offers centralized control over encryption keys with FIPS 140-2 validated HSMs, automatic rotation, detailed audit logging, and seamless integration across 100+ AWS services. Secrets Manager automates credential rotation, encrypts secrets at rest, versions all changes, and provides fine-grained access control through IAM and resource policies.

Key Takeaways

1. **Use Customer Managed CMKs for Production**

   Full control over policies, rotation, and deletion; required for compliance frameworks; $1/month per key is minimal cost for the control gained.

2. **Enable Automatic Key Rotation**

   Annual rotation is transparent to applications; maintains compliance; no code changes required. Best practice for all production keys.

3 **Implement Envelope Encryption**

Encrypt large data with local data keys; only the DEK is sent to KMS; eliminates the 4 KB limit and dramatically improves performance while reducing costs.

4 **Cache Secrets and Data Keys**

A 1-hour TTL reduces API calls by 99%; lowers costs from ~$50/month to ~$0.05/month; significantly improves application latency.

5 **Use Encryption Context**

Additional authenticated data prevents misuse; the context must match at decryption time; provides audit clarity with no additional cost.

6 **Automate Secret Rotation**

RDS/Redshift rotation is built-in; zero-downtime password changes; 30–90 day schedule recommended; eliminates manual rotation errors.

7 **Monitor with CloudTrail**

Every KMS and Secrets Manager API call is logged; use this to detect unauthorized access, produce compliance reports, and investigate incidents.

INTEGRATION ACROSS AWS

KMS and Secrets Manager integrate throughout AWS — encrypting S3 buckets, EBS volumes, RDS databases, Lambda environment variables, and SQS messages via KMS; rotating RDS credentials, storing API keys, and managing certificates via Secrets Manager. The next logical study area is **AWS CLOUDTRAIL AND CONFIG** for comprehensive audit logging and configuration compliance.